ASSIGNMENT: 10

Name: Rajdeep Das

Roll No: CSB25203

Subject: Advanced Programming

```python
# Address Class
class Address:
    def __init__(self, street, city, zip_code):
        self.street = street
        self.city = city
        self.zip_code = zip_code

    def __str__(self):
        return f"{self.street}, {self.city} - {self.zip_code}"


# Student Class (HAS-A Address → Composition)
class Student:
    def __init__(self, name, age, address, courses=None):
        self.name = name
        self._age = None  # protected attribute
        self.age = age    # use property setter
        self.address = address
        self.courses = courses if courses is not None else []

    # Property for age with validation
    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value <= 0 or value > 120:
            raise ValueError("Age must be between 1 and 120")
        self._age = value

    # Add course (mutable list behavior)
    def add_course(self, course):
        if not course:
            raise ValueError("Course name cannot be empty")
        self.courses.append(course)
```

```
    # Display details
    def display(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"Address: {self.address}")
        print(f"Courses: {', '.join(self.courses) if self.courses else
'None'}")


# Derived Class
class ScholarshipStudent(Student):
    def __init__(self, name, age, address, scholarship_amount, courses=None):
        super().__init__(name, age, address, courses)
        self.scholarship_amount = scholarship_amount

    # Override display()
    def display(self):
        super().display()
        print(f"Scholarship Amount: ₹{self.scholarship_amount}")


# Main Execution
if __name__ == "__main__":
    # Create Address (Composition)
    addr1 = Address("GS Road", "Guwahati", "781005")

    # Create Students
    student1 = Student("Rajdeep", 18, addr1)
    student2 = ScholarshipStudent("Aman", 20, addr1, 50000)

    # Add courses (mutable behavior)
    student1.add_course("Math")
    student1.add_course("Science")

    student2.add_course("Computer Science")

    # Polymorphism (same method, different behavior)
    students = [student1, student2]

    for s in students:
        print("\n--- Student Details ---")
        s.display()
```